Gazipur Digital University

Department of Software Engineering

Course Titel : **Object Oriented Programming**

Course Code : PROG112

Submitted To: Md. Moshiur Rahman
Lecturer & Chairman Department of Software Engineering

Submitted By : Md. Akash Mia
ID:2303039
Session:2023-2024

Date of Submission:4/09/25

Lab Report 2

Experiment No.: 2

Title: Object Oriented Programming using Java (Class & Object Implementation)

Q1: Online Shopping System

Objective:

To design a simple **Online Shopping System** in Java where a customer can place an order for a product, and the system will display the order details such as Customer Name, Product Name, Quantity, and Total Price.

Theory:

- **Class:** Blueprint for creating objects (Customer, Product, Order).
- **Object:** Instance of a class.
- **Constructor:** Special method to initialize objects.
- **Association:** Relationship between classes (Order has Customer and Product objects).

Here,

- **Product** → Holds productId, name, price.
- **Customer** → Holds customerId, name, email.
- **Order** → Holds orderId, customer, product, quantity. It calculates total price and displays details.

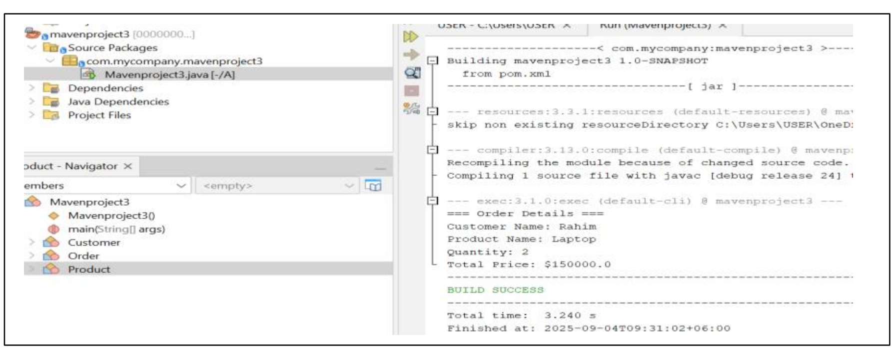
Source Code:

```
public class Mavenproject3 {
// Product class
    static class Product {
        int productId;
        String name;
        double price;

        Product(int productId, String name, double price) {
            this.productId = productId;
            this.name = name;
            this.price = price;
        }
    }
 // Customer class
    static class Customer {
        int customerId;
        String name;
        String email;

        Customer(int customerId, String name, String email) {
            this.customerId = customerId;
            this.name = name;
            this.email = email;
        }
    }
```

```
// Order class
   static class Order {
       int orderId;
       Customer customer;
       Product product;
       int quantity;

       Order(int orderId, Customer customer, Product product, int quantity) {
           this.orderId = orderId;
           this.customer = customer;
           this.product = product;
           this.quantity = quantity;
       }

       void displayOrderDetails() {
           double totalPrice = product.price * quantity;
           System.out.println("=== Order Details ===");
           System.out.println("Customer Name: " + customer.name);
           System.out.println("Product Name: " + product.name);
           System.out.println("Quantity: " + quantity);
           System.out.println("Total Price: $" + totalPrice);
       }
   }
// Main method
   public static void main(String[] args) {
       Customer customer = new Customer(1, "Rahim", "rahim@example.com");
       Product product = new Product(101, "Laptop", 75000.0);
       Order order = new Order(5001, customer, product, 2);

       order.displayOrderDetails();
   }
}
```

Output:

```
=== Order Details ===
Customer Name: Rahim
Product Name: Laptop
Quantity: 2
Total Price: $150000.0
```

.


Q2: University Management System

Objective:

To design a simple **University Management System** in Java where a student can enroll in a course, and the system will display the enrollment details such as Student Name, Course Name, and Semester.

Theory:

- **Student Class:** Contains studentId, name, department.
- **Course Class:** Contains courseId, courseName, credits.
- **Enrollment Class:** Links student and course with semester information.
- **Association:** Enrollment class associates a Student with a Course.

Source Code:

package com.mycompany.mavenproject2;

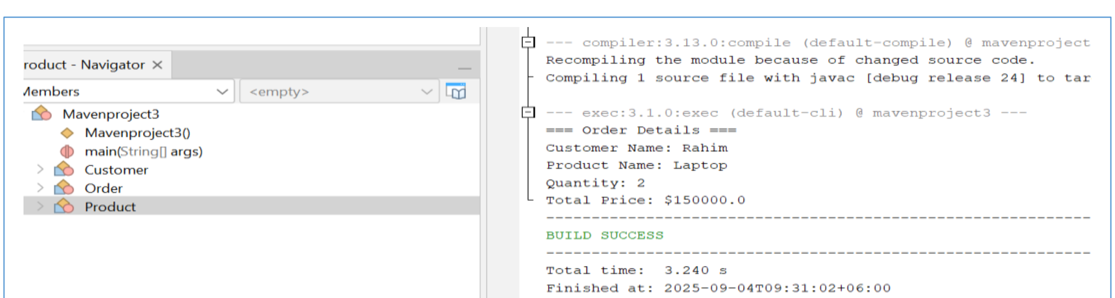
```
public class Mavenproject2 {

 // Student class

    static class Student {

        int studentId;

        String name;

        String department;

 Student(int studentId, String name, String department) {

            this.studentId = studentId;
```

```
            this.name = name;

            this.department = department;

        }

    }

// Course class

    static class Course {

        int courseId;

        String courseName;

        int credits;

   Course(int courseId, String courseName, int credits) {

            this.courseId = courseId;

            this.courseName = courseName;

            this.credits = credits;

        }

    }

    // Enrollment class

    static class Enrollment {

        int enrollmentId;

        Student student;

        Course course;

        String semester;

            Enrollment(int enrollmentId, Student student, Course course, String semester) {

            this.enrollmentId = enrollmentId;
```

```
            this.student = student;

            this.course = course;

            this.semester = semester;

        }

        void displayEnrollmentDetails() {

            System.out.println("=== Enrollment Details ===");

            System.out.println("Student Name: " + student.name);

            System.out.println("Course Name: " + course.courseName);

            System.out.println("Semester: " + semester);

        }

    }

 // Main method

    public static void main(String[] args) {

        Student student = new Student(1, "Karim", "Software Engineering");

        Course course = new Course(101, "Object Oriented Programming", 3);

        Enrollment enrollment = new Enrollment(1001, student, course, "Fall 2025");

   enrollment.displayEnrollmentDetails();

    }

}
```

Output:

```
=== Enrollment Details ===
Student Name: Karim
Course Name: Object Oriented Programming
Semester: Fall 2025
```



Conclusion:

This program demonstrates how objects of different classes can be associated to represent a real-world scenario. The Enrollment class connects Student and Course and shows the enrollment details.